

SDTS FOR VARIOUS LANGUAGE FEATURES

Infix to Postfix

$E \rightarrow T R$
 $R \rightarrow \text{addop } T \{ \text{print}(\text{addop.lexeme}) \} R_1 \mid \epsilon$
 $T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$

Declaration Processing

$P \rightarrow D$
 $D \rightarrow D; D$
 $D \rightarrow T L; \quad \{ \text{enter} (L.\text{list}, T.\text{type}, T.\text{width}, \text{offset})$
 $L \rightarrow L_1, \text{id} \quad \{ L.\text{list} := \text{append}(L_1.\text{list}, \text{id.name}) \}$
 $L \rightarrow \text{id} \quad \{ L.\text{list} = \text{makelist} (\text{id.name}) \}$
 $T \rightarrow \text{integer} \quad \{ T.\text{type} := \text{integer}; T.\text{width} := 4 \}$
 $T \rightarrow \text{real} \quad \{ T.\text{type} := \text{real}; T.\text{width} := 8 \}$
 $T \rightarrow \text{id} [\text{num}] \text{ of } T_1 \{ T.\text{width} := \text{num.val} * T_1.\text{width};$
 $\quad \quad \quad T.\text{type} := \text{array} (\text{num.val}, T_1.\text{type}) \}$
 $T \rightarrow \uparrow T_1 \quad \{ T.\text{type} := \text{pointer} (T_1.\text{type}); T.\text{width} := 4 \}$

Alternate SDTS

$D \rightarrow D; D$
 $D \rightarrow D_1, \text{id} \quad \{ \text{enter} (\text{id.name}, D_1.\text{syn}); D.\text{syn} := D_1.\text{syn} \}$
 $D \rightarrow T \text{ id} \quad \{ \text{enter} (\text{id.name}, T.\text{type}); D.\text{syn} := T.\text{type} \}$
 $T \rightarrow \text{integer} \mid \text{real} \mid \dots \quad \{ \text{same as earlier} \}$

SDTS for Procedure Declarations

We use the earlier grammar for illustration but add another rule to it.

$P \rightarrow D$
 $D \rightarrow D; D$
 $D \rightarrow \text{id} : T$
 $D \rightarrow \text{proc id} ; D ; S$
 $T \rightarrow \text{integer}$
 $T \rightarrow \text{real}$
 $T \rightarrow \text{array} [\text{num}] \text{ of } T_1$
 $T \rightarrow \uparrow T_1$

The rule, $D \rightarrow \text{proc id} ; D ; S$ permits nested procedures, but without parameters. The design solution is to create a separate table for handling names local to each procedure. The tables are linked in such a manner that lexical scoping rules are honoured.

SCOPE INFORMATION

For symbol table management, the following organization and routines are assumed.

1. Each table is an array of a suitable size which is linked using the fields as shown.
2. There are 2 stacks, called **tableptr** and **offset**. The stack **tableptr** points to the currently active procedure's table and the pointers to the tables of the enclosing procedures are kept below in the stack.
3. The other stack is used to compute the relative addresses for locals within a procedure.
4. **mktable(previous)** is a procedure that creates a new table and returns a pointer to it. It also gives the pointer to the previous table through the argument, **previous**. This is required in order to link the new table with that of the closest enclosing procedure's.
5. **enter(table, name, type, width, offset)** is a procedure for creating a new entry for the local variable, name, along with its attributes.

6. **enter**(table, name, type, width, offset) is a procedure for creating a new entry for the local variable, name, along with its attributes.
7. **addwidth**(table,width) is used to compute the total space requirement for all the names of a procedure.
8. **enterproc** (table, name, nest level, newtable) creates a new entry for a procedure name. The last argument is a pointer to the corresponding table.

The semantic rules are given now.

- $$\begin{aligned}
 P &\rightarrow M D \quad \{ \text{addwidth}(\text{top}(\text{tableptr}), \text{top}(\text{offset})) ; \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset}) \} \\
 M &\rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{nil}); \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}); \text{nest} := 1 \} \\
 D &\rightarrow D ; D \\
 D &\rightarrow \text{id} : T \quad \{ \text{enter}(\text{top}(\text{tableptr}), \text{id.name}, T.\text{type}, T.\text{width}); \text{top}(\text{offset}) := \text{top}(\text{offset}) + T.\text{width} \} \\
 D &\rightarrow \text{proc id} ; N D ; S \quad \{ p := \text{top}(\text{tableptr}); \text{addwidth}(p, \text{top}(\text{offset})); \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset}); \\
 &\quad \text{enterproc}(\text{top}(\text{tableptr}), \text{id.name}, \text{nest}, p); \} \\
 N &\rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{top}(\text{tableptr})); \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}) ; \text{nest}++; \} \\
 T &\rightarrow \text{integer} \quad \{ T.\text{type} := \text{integer}; T.\text{width} := 4 \} \\
 T &\rightarrow \text{real} \quad \{ T.\text{type} := \text{real}; T.\text{width} := 8 \} \\
 T &\rightarrow \text{id} [\text{num}] \text{ of } T_1 \quad \{ T.\text{width} := \text{num.val} * T_1.\text{width}; T.\text{type} := \text{array}(\text{num.val}, T_1.\text{type}) \} \\
 T &\rightarrow \uparrow T_1 \quad \{ T.\text{type} := \text{pointer}(T_1.\text{type}); T.\text{width} := 4 \}
 \end{aligned}$$
- The translation scheme described is inadequate to handle recursive procedures. The reason being that the name of such a procedure would be entered in its closest enclosing symbol table only after the entire procedure is parsed.
 - However a recursive procedure would have a call to itself within its body and this call cannot be translated since the procedure name would not be present in the table.
 - A possible remedy is to enter a procedure's name immediately after it has been found. Semantic actions for only the changed rules are shown.

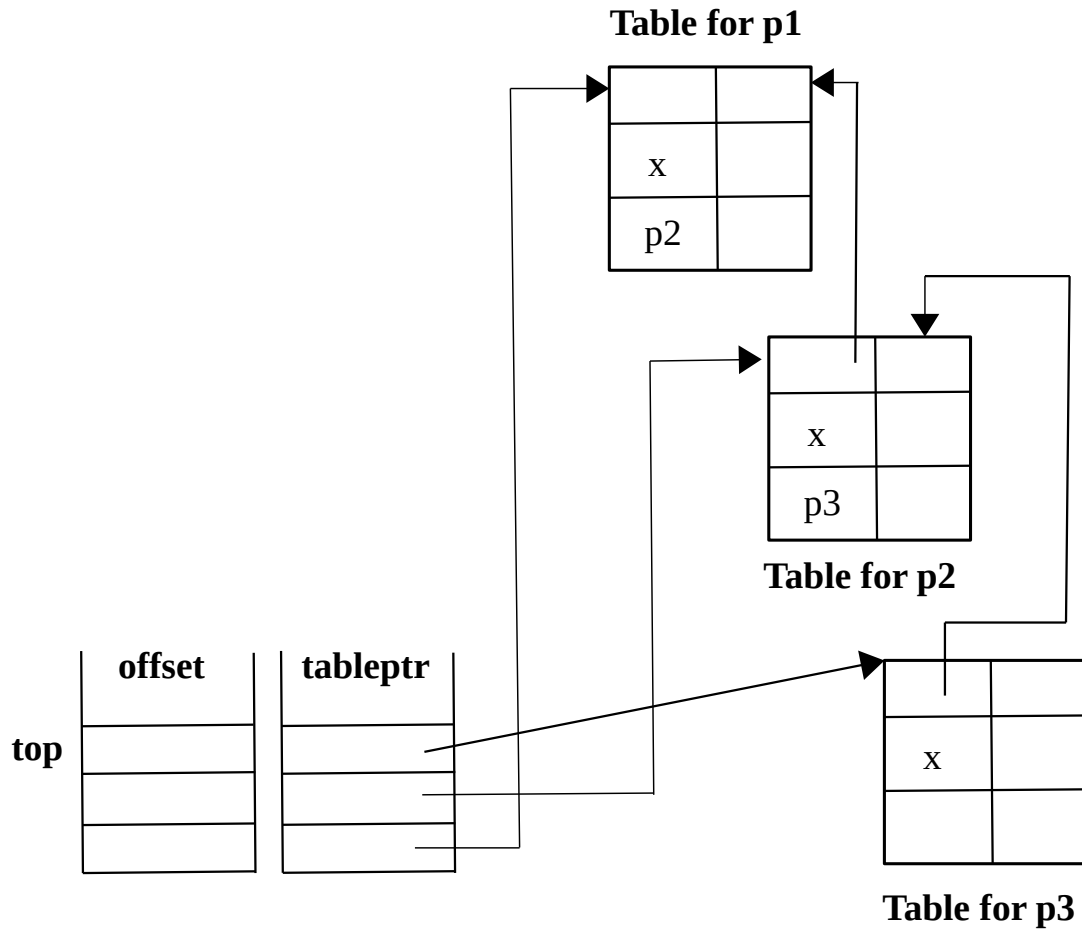
$$\begin{aligned}
 D &\rightarrow \text{proc id} ; N D ; S \\
 &\quad \{ p := \text{top}(\text{tableptr}); \text{addwidth}(p, \text{top}(\text{offset})); \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset}); \text{nest}--; \} \\
 N &\rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{top}(\text{tableptr})); \text{enterproc}(\text{top}(\text{tableptr}), \text{id.name}, \text{nest}, p) \\
 &\quad \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}) ; \text{nest}++; \}
 \end{aligned}$$

Processing of Record Data type

$$\begin{aligned}
 D &\rightarrow D ; D \mid \text{id} : T \\
 T &\rightarrow \text{integer} \mid \text{real} \\
 T &\rightarrow \text{id} [\text{num}] \text{ of } T_1 \\
 T &\rightarrow \text{record } L D \text{ end} \\
 L &\rightarrow \varepsilon
 \end{aligned}$$

Two stacks tableptr and offset are used in the same sense as introduced earlier. Attribute T.width gives the width of all the data objects declared within a record. Attribute T.type is used to hold the type of a record.

$$\begin{aligned}
 T &\rightarrow \text{record } L D \text{ end} \\
 &\quad \{ T.\text{type} := \text{record}(\text{top}(\text{tableptr}); T.\text{width} := \text{top}(\text{offset}); \\
 &\quad \quad \text{pop}(\text{tableptr}); \text{pop}(\text{offset}) \\
 &\quad \} \\
 L &\rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{nil}); \\
 &\quad \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}) \\
 &\quad \}
 \end{aligned}$$

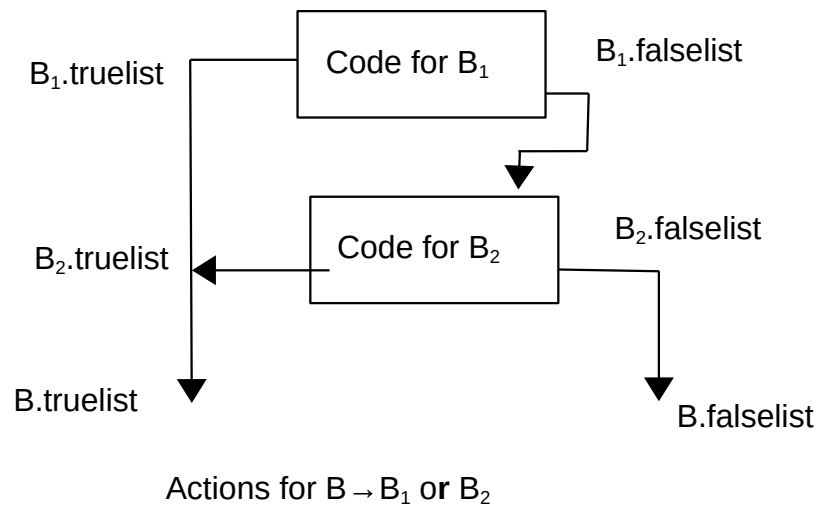
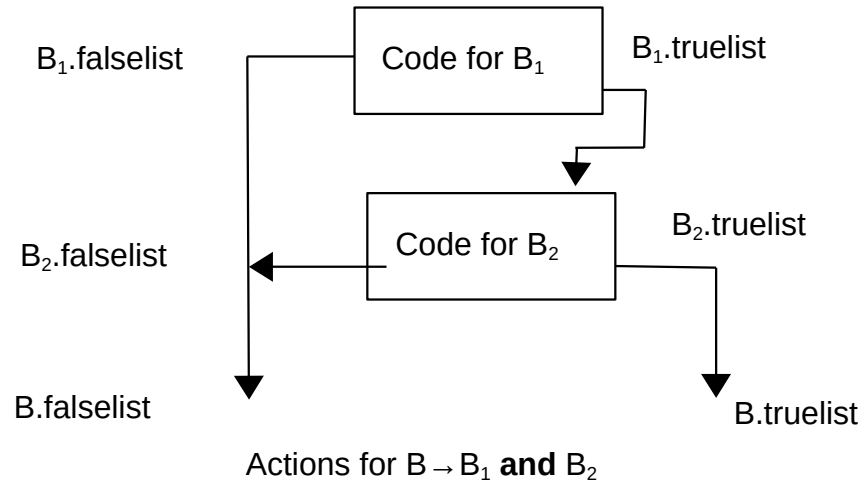


(c) Symbol Tables and their interconnection when control reaches the marked point

Semantic errors that are detected during processing of declarations of a program include the following among many others

- uniqueness checks
- names referenced but not declared
- names declared but not referenced
- names declared but not initialized properly
- ambiguity in declarations
- and many more

Code for detection and handling of such errors are incorporated in the semantics actions at the relevant places.



Translation of Basic Assignment Statement

```
S → id := E      { p := lookup (id.name); if p ≠ nil then emit ( p ':=' E.place) else error /* undeclared id */ }
E → E1 + E2    { E.place := newtemp(); emit (E.place ':=' E1.place '+' E2.place)}
E → E1 * E2    { E.place := newtemp(); emit (E.place ':=' E1.place '*' E2.place)}
E → - E1       { E.place := newtemp(); emit (E.place ':=' 'uminus' E1.place) }
E → ( E1 )     { E.place := E1.place }
E → id          { p := lookup (id.name); if p ≠ nil then E.place := p else error /* undeclared id */ }
```

Translation of Assignment Statement with Type Conversion

```
E → E1 + E2 { if E1.type = integer and sequiv( E1.type, E2.type)
    then {E.place := newtemp_int(); emit(E.place ':=' E1.place 'int +' E2.place); E.type := integer}
    else if E1.type = real and sequiv( E1.type, E2.type)
        then { E.place := newtemp_real(); emit(E.place ':=' E1.place 'real +' E2.place); E.type := real}
    else if E1.type = real and E2.type = integer
        then {E.place := newtemp_real(); t := newtemp_real(); emit(t ':=' 'inttoreal' E2.place);
            emit( E.place ':=' E1.place 'real +' t); E.type := real }
    else if E1.type = integer and E2.type = real
        then {E.place := newtemp_real(); t := newtemp_real(); emit(t ':=' 'inttoreal' E1.place);
            emit( E.place ':=' t 'real +' E2.place); E.type := real }
    else E.type := type_error
}
```

```
S → id := E { p := lookup (id.name) ; if p = nil then error /* undeclared id */
    else if sequiv (type_of(p), E.type) then if E.type = integer
        then {emit ( p 'int:=' E.place) ; S.type := void}
        else { emit ( p 'real:=' E.place); S.type := void }
    else if type_of (p) = real and E.type = integer
        then { t := newtemp_real (); emit ( t ':=' 'inttoreal' E.place); emit ( p 'real:= ' t ) ;
            S.type := void; }
    else S.type := type error; /* type mismatch */ }
```

Note that the temporaries used above are of different types.

Array References (Syntax and semantics of C/C++)

Address Calculations

Address calculation at compile time for an integer array, $a[10,20]$, to find the **address of $a[i_1, i_2]$**

Row major representation with base $a[1, 1]$	Row major representation with base $a[0, 0]$
Let base be the relative address of $a[1,1]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 - 1) * 20 + i_2 - 1) * w$ $= c + v$ where $c = \text{base} - 21 * w$ is a compile time constant and $v = (20 * i_1 + i_2) * w$ is a function of i_1, i_2	Let base be the relative address of $a[0,0]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 * 20 + i_2) * w$ $= c + v$ where $c = \text{base}$ is a compile time constant and $v = (20 * i_1 + i_2) * w$ is a function of i_1, i_2

Similarly we perform address calculations when column-major representation is used.

Column major representation with base $a[1, 1]$	Column major representation with base $a[0, 0]$
Let base be the relative address of $a[1,1]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 - 1) * 10 + i_2 - 1) * w$ $= c + v$ where $c = \text{base} - 11 * w$ is a compile time constant and $v = (10 * i_1 + i_2) * w$ is a function of i_1, i_2	Let base be the relative address of $a[0,0]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 * 10 + i_2) * w$ $= c + v$ where $c = \text{base}$ is a compile time constant and $v = (10 * i_1 + i_2) * w$ is a function of i_1, i_2

Assume an array reference is of the form : $a[e_1, e_2, \dots, e_n]$ in the context of C / C++.

Expression seen by the Parser	Attributes available	Code Generated
$a[e_1,$	$c = \text{addr}(a)$ expression e_1	$t1 \leftarrow \text{evaluate}(e_1)$
$e_2,$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1,$	$t1 := t1 * u_2$ $t2 \leftarrow \text{evaluate}(e_2)$ $t1 := t1 + t2$
$e_3,$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2,$	$t1 := t1 * u_3$ $t2 \leftarrow \text{evaluate}(e_3)$ $t1 := t1 + t2$
$\dots$	$\dots$	$\dots$
$e_{n-1},$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2, e_3, \dots, e_{n-2},$	$t1 := t1 * u_n$ $t2 \leftarrow \text{evaluate}(e_{n-1})$ $t1 := t1 + t2$
$e_n]$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2, e_3, \dots, e_{n-1},$	$t2 \leftarrow \text{evaluate}(e_n)$ $t1 := t1 + t2$ $t1 := t1 * w$ $t0 := c$ $t0[t1]$

After processing of $a[e_1, e_2, \dots, e_n]$, this array reference will be replaced by the compiler as $t_0[t_1]$ in its intermediate code in whatever context this array reference occurred in the source.

Note that this scheme attempts to reuse temporaries, as parsing proceeds, and overall only two temporary variables are in use at any given point, eventually saving the result using 2 temporaries, t0 and t1. In view of the translation scheme outlined above, we need to write a grammar that exposes the index expressions one by one so that the parser can perform syntax and semantic analysis hand in hand.

```

S → L = E { if L.offset = null then emit (L.place ':=' E.place)
             else emit (L.place '[' L.offset ']' ':=' E.place) }
E → L { if L.offset = null then E.place := L.place
        else E.place := newtemp(); emit (E.place ':=' L.place '[' L.offset ']) }
L → id { L.place := id.place; L.offset := null }
L → elist ] { L.place := newtemp(); L.offset := newtemp(); emit(L.place ':=' base(elist.array));
             emit(L.offset ':=' elist.place '*' width(elist.array)) }
elist → elist1 [ E { m := elist1.dim + 1 ; emit( elist1.place ':=' elist1.place '*' limit(elist1.array, m));
                  emit ( E.place ':=' E.place '+' elist1.place );
                  elist.array := elist1.array; elist.place := E.place ; elist.dim := m }
elist → id [ E { elist.array := id.place; elist.place := E.place; elist.dim := 1 }

```

Exercise 1 : Formulate the recurrence relations for multidimensional array references permitted in C / C++. Note that the syntax for array reference in C / C++ is different than the form used here.

Exercise 2 : Formulate the recurrence relations for multidimensional array references that are permitted in Fortran like languages that use column major representation with lower bounds of all dimensions starting at 1.

Exercise 3 : Formulate the recurrence relations for multidimensional array references permitted in some languages, which use row major representation but permit negative indices. In general, they allow array declarations of the form :

integer a[L₁:U₁, L₂:U₂, ..., L_n:U_n];

where both L_i and U_i may be negative, but L_i ≤ U_i, 1 ≤ i ≤ n.

In general, given a representation, row-major or column major, the lower and upper bounds, a compiler can always determine how to generate intermediate code along with parsing as each relevant parts of the reference are encountered.

SDTS for Full Evaluation of Boolean expressions is now given.

```

B → B1 or B2 { B.place := newtemp(); emit(B.place ':=' B1.place 'or' B2.place) }
B → B1 and B2 { B.place := newtemp(); emit(B.place ':=' B1.place 'and' B2.place) }
B → not B1 { B.place := newtemp(); emit(B.place ':=' 'not' B1.place) }
B → ( B1 ) { B.place := B1.place }
B → true { B.place := newtemp(); emit (B.place ':=' '1') }
B → false { B.place := newtemp(); emit (B.place ':=' '0') }
B → E { B.place := E.place }
B → E1 relop E2 { B.place := newtemp(); emit ('if' E1.place 'relop.op' E2.place 'goto' nextstat+3);
                  emit (B.place ':=' '0'); emit ('goto' nextstat + 2); emit (B.place ':=' '1')}

```

Scheme2 : Partial evaluation of boolean expressions :

```

B → B1 or M B2 { backpatch (B1.falselist, M.stat); B.truelist := merge (B1.truelist, B2.truelist);
                  B.falselist := B2.falselist }
B → B1 and M B2 { backpatch (B1.truelist, M.stat); B.falselist := merge (B1.falselist, B2.falselist);
                  B.truelist := B2.truelist }
B → not B1 { B.truelist := B1.falselist ; B.falselist := B1.truelist }
B → ( B1 ) { B.truelist := B1.truelist ; B.falselist := B1.falselist }
B → true { B.truelist := makelist (nextstat); emit ('goto ____');
B → false { B.falselist := makelist (nextstat); emit ('goto ____');

```

$B \rightarrow E_1 \text{ relop } E_2 \{ B.\text{truelist} := \text{makelist}(\text{nextstat}); B.\text{falselist} := \text{makelist}(\text{nextstat} + 1);$
 $\quad \text{emit}('if' E_1.\text{place} \text{ 'relop.op' } E_2.\text{place} \text{ 'goto ___'}); \text{emit}('goto ___') \}$
 $B \rightarrow E \{ B.\text{place} := E.\text{place}; B.\text{truelist} := \text{makelist}(\text{nextstat}); B.\text{falselist} := \text{makelist}(\text{nextstat} + 1);$
 $\quad \text{emit}('if' B.\text{place} \text{ 'goto ___'}); \text{emit}('goto ___') \}$
 $M \rightarrow \epsilon \{ M.\text{stat} := \text{nextstat} \}$
 $E \rightarrow E_1 + E_2 \mid \text{other rules for expressions} \{ \text{semantic actions as discussed earlier} \}$

4. SDTS for control flow constructs

$S \rightarrow \text{if } B \text{ then } M S_1$
 $\quad \{ \text{backpatch}(B.\text{truelist}, M.\text{stat}); S.\text{nextlist} := \text{merge}(B.\text{falselist}, S_1.\text{nextlist}) \}$
 $S \rightarrow \text{if } B \text{ then } M_1 S_1 N \text{ else } M_2 S \{ \text{backpatch}(B.\text{truelist}, M_1.\text{stat}); \text{backpatch}(B.\text{falselist}, M_2.\text{stat});$
 $\quad S.\text{next} := \text{merge}(S_1.\text{nextlist}, \text{merge}(N.\text{nextlist}, S_2.\text{nextlist})) \}$
 $S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1 \{ \text{backpatch}(B.\text{truelist}, M_2.\text{stat}); \text{backpatch}(S_1.\text{nextlist}, M_1.\text{stat});$
 $\quad S.\text{nextlist} := B.\text{falselist}; \text{emit}('goto' M_1.\text{stat}) \}$
 $S \rightarrow \text{begin } S_l \text{ end} \{ S.\text{nextlist} := S_l.\text{nextlist} \}$
 $S_l \rightarrow S_{l1}; M S \{ \text{backpatch}(S_{l1}.\text{nextlist}, M.\text{stat}); S_l.\text{nextlist} := S.\text{nextlist} \}$
 $S_l \rightarrow S \{ S_l.\text{nextlist} := S.\text{nextlist} \}$
 $S \rightarrow L := E \{ \text{semantic actions as earlier for assignment}; S.\text{nextlist} := \text{nil} \}$
 $M \rightarrow \epsilon \{ M.\text{stat} := \text{nextstat} \}$
 $N \rightarrow \epsilon \{ N.\text{nextlist} := \text{makelist}(\text{nextstat}); \text{emit}('goto ___') \}$

5. Translation of Other constructs : The basic issues in semantic analysis and translation have been covered. However, a compiler has to deal with more details, a few of which are listed below. Translation of goto statement or break statements. It is handled by storing labels in the symbol table, associating index of 3-address code with the label and using backpatching to resolve forward references.

Procedure call statement

Let parameter passing be call by reference and storage be statically allocated.

- Semantic analysis involves type checking of the formal and actual arguments.
- Translation would require generating 3-address code for evaluating each argument (in case it is an expression). Then generate a list of 3-address **param** statements, one for each argument, followed by the 3-address call statement.

Translation of other constructs such as **case** statement, **for** statement and **repeat - until** statement can be worked out on similar lines.